A Proof of the Cochain-Complex Condition

Let the node-to-edge maps be parameterized as in Equation (10). For every path instance constructed by H2S-NN whose node sequence is a simple path and whose boundary is enumerated forward, the maps in Equations (19)–(20) satisfy $\delta_{1,w}\delta_0 = 0$ identically for every node cochain and every value of the learned parameters, with no condition on Θ_f.

Proof. By construction $F_{e_i \trianglelefteq f} F_{u \trianglelefteq e_i} = T_{v_{i-1}}$ and $F_{e_i \trianglelefteq f} F_{v \trianglelefteq e_i} = T_{v_i}$. The i-th path term of $\delta_{1,w}\delta_0 x$ is therefore $T_{v_i} x_{v_i} - T_{v_{i-1}} x_{v_{i-1}}$, and summing over the path telescopes to $T_{v_\ell} x_{v_\ell} - T_{v_0} x_{v_0}$. The closure edge is stored as $e_{\mathrm{cl}} = (v_\ell \to v_0)$, so $(\delta_0 x)_{e_{\mathrm{cl}}} = F_{v_0 \trianglelefteq e_{\mathrm{cl}}} x_{v_0} - F_{v_\ell \trianglelefteq e_{\mathrm{cl}}} x_{v_\ell}$, and the closure maps give $F_{e_{\mathrm{cl}} \trianglelefteq f} F_{v_0 \trianglelefteq e_{\mathrm{cl}}} = T_{v_0}$ and $F_{e_{\mathrm{cl}} \trianglelefteq f} F_{v_\ell \trianglelefteq e_{\mathrm{cl}}} = \Theta_f H_f = T_{v_\ell}$, so the closure term contributes $T_{v_0} x_{v_0} - T_{v_\ell} x_{v_\ell}$. The total sum is zero. The simple-path condition is required because the transport assigns one map to each node position; a revisited node, including a coincident endpoint pair, would require two transported values for the same node. □

This makes $C^0 \to C^1 \to C^2$ a cochain complex, so $\operatorname{im}\delta_0 \subseteq \ker \delta_{1,w}$: the face-induced component annihilates every edge cochain that is a coboundary of node states, and it acts only on the part of s that carries genuinely edge-level information. This is why Equation (12) initializes $s^{(0)}$ independently of $\delta_0 h^{(0)}$, and it holds for every value of Θ_f, so the face encoder is unconstrained.

B Task-Specific Readouts

After sheaf-Hodge diffusion, the model has integrated information across both nodes and typed relations while retaining representations at the node and edge levels. Different prediction tasks, however, require this information to be summarized differently: node classification is centered on an individual target node, whereas link prediction and graph classification also depend on relational or graph-level structure. We therefore keep the sheaf-Hodge backbone shared across tasks and use a lightweight task-specific readout to convert the relevant final representations into logits.

Specifically, the backbone produces final node states $x^\star$, final edge states $s^\star$, and the initial typed node states $h^{(0)}$, which the corresponding decoder uses according to the downstream task.

Node classification. For node classification, each incident edge state is first transported into the stalk of its incident node before aggregation. Let $V_{\mathrm{target}} \subseteq V$ denote the target-node set. For each node v, define

$$\widetilde{s}_v^\star = \sum_{e:\mathrm{src}(e)=v} F_{v \trianglelefteq e}^\top s_e^\star + \sum_{e:\mathrm{tgt}(e)=v} F_{v \trianglelefteq e}^\top s_e^\star. \tag{31}$$

The transposed restriction maps transport each edge state from its edge stalk into the corresponding node stalk before summation. Both endpoint contributions use a positive sign because $\widetilde{s}_v^\star$ is an incident-edge context summary rather than the orientation-sensitive coboundary adjoint $\delta_0^\top s^\star$. The node logits are then

$$\hat{y}_v = \mathrm{MLP}_{\mathrm{node}}\left(x_v^\star \,\|\, h_v^{(0)} \,\|\, \widetilde{s}_v^\star\right), \qquad v \in V_{\mathrm{target}}. \tag{32}$$

Link prediction. For link prediction, we reuse the transported context $\widetilde{s}_v^\star$ from Equation (31) and concatenate it with the final node state:

$$z_v = \mathrm{GELU}\big(W_{\mathrm{link}}[x_v^\star \,\|\, \widetilde{s}_v^\star] + b_{\mathrm{link}}\big) \in \mathbb{R}^d.$$

This readout makes the link-prediction loss depend on the edge cochain while combining only vectors expressed in the same endpoint node stalk. Given a candidate pair (u, v), the dot-product decoder returns

$$\hat{y}_{uv}^{\mathrm{dot}} = z_u^\top z_v. \tag{33}$$

Graph classification. For graph classification, SheafCovPool converts the node and edge cochains of a graph g into a summary vector γ_{g}. The attention-weighted means and covariances of the Gram-matrix features make this vector invariant to node and edge permutations and to independent orthogonal changes of stalk basis. The graph classifier is then a final MLP:

$$\hat{y}_{\mathrm{g}} = \mathrm{MLP}_{\mathrm{graph}}(\gamma_{\mathrm{g}}). \tag{34}$$

The logits $\hat{y}_{\mathrm{g}}$ are trained with cross-entropy for single-label graph classification, or binary cross-entropy with logits for multilabel graph classification.

C Hyperparameter Tuning

We performed hyperparameter tuning with Weights & Biases (W&B) Bayesian optimization and Hyperband early termination, taking the task-specific validation metric as the optimization objective. For node classification and link prediction, each epoch consisted of a single full-graph forward and backward pass. For graph classification, the batch size was treated as a tunable hyperparameter selected from $\{32, 64, 128, 256, 512, 1024\}$.

D Benchmark Datasets

We evaluated the models on benchmark datasets covering node classification, link prediction, and graph classification. The node-classification and link-prediction datasets are taken the Heterogeneous Graph Benchmark (HGB) [23]. For graph classification, we used datasets from TUDataset [26]; node and edge labels are treated as node and relation types to instantiate heterogeneous graph inputs [3].

D.1 Node Classification

DBLP.. DBLP is a bibliographic network containing four types of entities: authors, papers, venues, and terms. These entities are connected through six relation types, including authorship, publication-venue, and paper-term relations. The prediction target is the author node type, and each author is assigned to one of four research areas: database systems, data mining, artificial intelligence, or information retrieval.

ACM.. ACM is an academic citation network with four node types: papers, authors, subjects, and terms. Its relations include paper citations, authorship, paper-subject associations, and paper-term associations. The objective is to classify papers into three research categories: database systems, wireless communication, and data mining.

IMDB.. IMDB is a heterogeneous movie network composed of movies, directors, actors, and keywords. Its six relation types include movie-director, movie-actor, and movie-keyword associations.

Table 6: Hyperparameter search space.

Type	Parameter	Search space
H2S-NN	Stalk dimension	node/link: {16, 32}; graph: {16, 32, 64}
	Hodge-diffusion steps K	{2, 3, 4, 5, 6}
	Node-cochain step size α_0	log-uniform on [0.005, 0.3]
	Edge-cochain step size α_1	log-uniform on [0.005, 0.3]
	Layer dropout	{0.0, 0.1, 0.2, 0.3}
	Input dropout	{0.0, 0.1, 0.2, 0.3}
	Classifier/head dropout	{0.0, 0.1, 0.2, 0.3} when consumed by the head
	Learning rate	$\{10^{-5}, 5{\times}10^{-5}, 10^{-4}, 5{\times}10^{-4}, 10^{-3}, 5{\times}10^{-3}, 10^{-2}, 5{\times}10^{-2}\}$
	Weight decay	$\{0, 10^{-5}, 5{\times}10^{-5}, 10^{-4}, 5{\times}10^{-4}, 10^{-3}, 5{\times}10^{-3}\}$
SheafCovPool	Raw Gram dimension	fixed at 3×3 (three semantic views)
	Post-Gram feature dimension d_p	{4, 8, 16}

Table 7: Node-classification dataset statistics and semantic meta-path schemas. The #N and #E columns indicate nodes and edges.

Dataset	#N	#N types	#E	#R types	#Classes	Target	Meta-path schemas
DBLP	26,128	4	239,566	6	4	author	APA, APCPA, APTPA
ACM	10,942	4	547,872	8	3	paper	PAP, PSP
IMDB	21,420	4	86,642	6	5	movie	MAM, MDM

The task is multilabel movie classification, in which a movie may belong to one or more of five genres: action, comedy, drama, romance, and thriller.

D.2 Link Prediction

Table 8: Heterogeneous link-prediction dataset statistics. The #N and #E columns indicate nodes and edges.

Dataset	#N	#N types	#E	#R types	Target
LastFM	20,612	3	141,521	3	user-artist
PubMed	63,109	4	244,986	10	disease-disease
YouTube	2,000	1	1,177,130	5	user-user
Amazon	10,099	1	148,659	2	product-product

LastFM.. LastFM is a music-oriented heterogeneous network containing users, artists, and tags. The graph includes user-artist, user-user, and artist-tag relations. The link-prediction task is to determine whether an interaction exists between a user and an artist.

PubMed. PubMed is a biological knowledge graph containing genes, diseases, chemicals, and species. Ten relation types describe biological associations, including gene-disease, chemical-chemical, and species-disease relations. Evaluation focuses on predicting links between disease nodes.

YouTube. YouTube is a social interaction graph containing 2,000 user nodes connected by five distinct types of user interactions. The prediction task is to infer missing user-user links.

Amazon. Amazon is a product interaction graph with one node type and two relation types. The task is to predict missing product-product links.

D.3 Graph Classification

Table 9: Graph-classification dataset statistics. The #Graphs column indicates the number of graphs; the #N and #E columns give the numbers of nodes and stored directed edges summed over all graphs in the dataset.

Dataset	#Graphs	#N	#N types	#E	#R types	Target
MUTAG	188	3,371	7	7,442	4	mutagenicity
PROTEINS	1,113	43,471	3	162,088	1	enzyme/non-enzyme
ENZYMES	600	19,580	3	74,564	1	enzyme class
NCI1	4,110	122,747	37	265,506	1	anti-cancer activity

MUTAG.. MUTAG contains 188 molecular graphs representing aromatic and heteroaromatic nitro compounds. Nodes correspond to seven atom types, while edges encode four bond types. Each graph is classified according to whether the corresponding compound exhibits a mutagenic effect.

PROTEINS.. PROTEINS consists of 1,113 protein graphs. Nodes represent secondary-structure elements belonging to three types, and edges connect elements that are adjacent either in the amino-acid sequence or in three-dimensional space. The task is binary classification of proteins into enzyme and non-enzyme classes.

ENZYMES.. ENZYMES contains 600 graph representations of protein tertiary structures collected from the BRENDA enzyme database. Its nodes describe three types of secondary-structure elements. Each graph is assigned to one of the six top-level Enzyme Commission classes.

NCI1. NCI1 comprises 4,110 chemical compounds evaluated for their ability to inhibit the growth of human tumor cell lines. Molecular nodes represent 37 atom types. The graph-classification objective is to determine whether each compound exhibits anti-cancer activity.

E Additional H2S-NN Experiments: Node Clustering

Although the main experiments evaluated supervised graph-learning tasks, we also used clustering to examine whether the learned representations preserve class-relevant local structure. Following the evaluation procedure used for the Heterogeneous Graph Attention Network [38], we ran k-means on the node embeddings after supervised training. We used DBLP with $k = 4$ and ACM with $k = 3$, matching the number of classes in each dataset, and set the maximum number of iterations to 300.

We compared H2S-NN against four groups of baselines: (i) homogeneous homophilic model, namely GAT [36]; (ii) homogeneous heterophilic models, including NSD [2] and CSNN [29]; (iii) heterogeneous homophilic models, including HAN [38] and PSHGCN [14]; and (iv) heterogeneous heterophilic models, including Hetero^{2}Net [19] and LatGRL [30].

We used normalized mutual information (NMI) and the adjusted Rand index (ARI) to evaluate clustering quality. As shown in Table 10, H2S-NN achieved the highest mean NMI and ARI on both DBLP and ACM. These results suggest that the embeddings learned by H2S-NN are more separable under this clustering diagnostic. The t-SNE [35] visualizations in Figures 5 and 6 provide a qualitative view of these embeddings.



Figure 5: t-SNE visualization of learned ACM paper embeddings. Each point represents one paper, and color denotes its class.

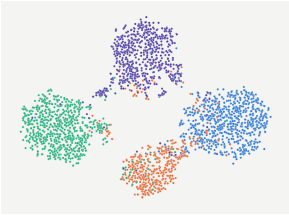


Figure 6: t-SNE visualization of learned DBLP author embeddings. Each point represents one author, and color denotes its research area.

F Visualization of Node Embedding Separability During Training

We further inspected the training dynamics of H2S-NN on the controlled four-class `FakeHeteroDataset` from PyTorch Geometric [7, 8]. The model was trained with the default synthetic-data settings, and the visualization includes only labeled nodes. We recorded the node embeddings every ten epochs. As shown in Figure 7, nodes from different classes are strongly interleaved at the beginning of training. As training proceeds, the projected embeddings of nodes in the same class form increasingly compact and separated regions. This observation suggests that training H2S-NN produces more class-separable node representations on this controlled dataset.

Table 10: Additional clustering-quality diagnostics on DBLP and ACM, reported as mean ± standard deviation over five runs. Best mean values are bold.

		DBLP		ACM	
Group	**Model**	NMI	ARI	NMI	ARI
Hom./hom.	GAT	71.50 ± 0.36	77.26 ± 0.48	57.22 ± 0.19	60.47 ± 0.31
Hom./het.	NSD	80.21 ± 0.62	84.04 ± 0.43	62.56 ± 0.37	65.02 ± 0.28
	CSNN	81.72 ± 0.39	84.25 ± 0.55	69.28 ± 0.45	71.10 ± 0.34
Het./hom.	HAN	79.12 ± 0.47	84.76 ± 0.58	60.63 ± 0.22	64.28 ± 0.41
	PSHGCN	80.38 ± 0.52	82.65 ± 0.36	71.89 ± 0.49	74.31 ± 0.57
Het./het.	Hetero^{2}Net	80.54 ± 0.33	84.98 ± 0.46	70.14 ± 0.40	73.42 ± 0.51
	LatGRL	81.36 ± 0.56	86.07 ± 0.38	72.52 ± 0.61	76.75 ± 0.44
Ours	**H2S-NN**	**82.41 ± 0.35**	**86.52 ± 0.42**	**75.78 ± 0.37**	**78.29 ± 0.31**

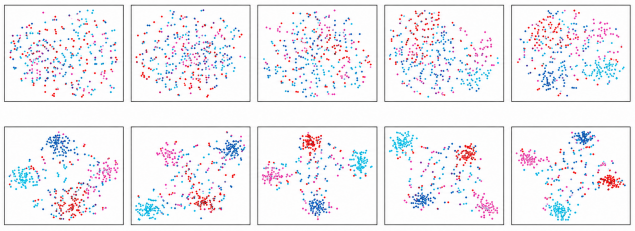


Figure 7: Qualitative visualization of node embedding dynamics during training on the four-class `FakeHeteroDataset`.

G Computational Efficiency

To characterize the computational cost, we analyzed the H2S-NN model, the AMPSS component, and the SheafCovPool readout.

Let $N = |V|$ and $M = |E|$ denote the numbers of nodes and observed directed edges. Let N_{inst} be the number of retained concrete path instances. Define the number of boundary incidences by

$$I_{\text{bnd}} = \sum_{f \in \mathcal{I}_{\text{ret}}} (\ell_f + 1).$$

Each instance contributes one closure 1-cell in addition to its path edges. Let $\widetilde{M} = M + N_{\text{inst}}$ denote the size of the extended edge domain. We use d for the stalk dimension, K for the number of diffusion steps, and $d_{\text{in}} = \max_{t \in \mathcal{T}} d_t$ for the widest raw node-feature dimension, where $x_v^{\text{raw}} \in \mathbb{R}^{d_{\phi(v)}}$ as in Section 3.

AMPSS.. Stage I runs once before training. Let C_Π be the number of viable schemas, K_S the schema limit, N_p the number of probe examples, d_{sch} the feature width per schema, C_{cls} the number of

classes, and K_{sel} the number of probe iterations. Greedy selection evaluates $O(C_\Pi K_S)$ subsets whose feature width grows to $K_S d_{\text{sch}}$, giving

$$O(K_{\text{sel}} N_p d_{\text{sch}} C_{\text{cls}} C_\Pi K_S^2)$$

probe time. Label propagation and schema/path enumeration additionally cost $O(K_{\text{LP}}(M+N)C_{\text{cls}} + N_{\text{type}} + N_{\text{samp}})$, where K_{LP} is the number of label-propagation iterations and N_{type} and N_{samp} count the visited typed and concrete path prefixes. Sparse coverage and schema features require

$$O(C_\Pi \ell_{\max} + N_p K_S d_{\text{sch}} + N_{\text{cov}} + I_{\text{bnd}})$$

storage, where $\ell_{\max}$ is the maximum schema length and N_{cov} is the number of stored coverage entries. Stage II adds $O(N_{\text{inst}}(d_\xi + d_r)d_w)$ instance-weighting time, where d_ξ is the ordered-path latent width and d_w is the weighting-network width. Because the weights are strictly positive, they reweight but never remove boundary incidences, so the operator's sparsity pattern is unchanged.

H2S-NN.. To construct the scaled-orthogonal restriction maps, we use the Fast Householder Product parameterization [25]. Node-to-edge maps are learned only for the M observed edges, whereas closure and edge-to-face maps are obtained by path transport. Their dense $d \times d$ matrix operations require $O((M+I_{\text{bnd}})d^3)$ construction time. The resulting maps are cached across diffusion steps, requiring $O((\widetilde{M}+I_{\text{bnd}})d^2)$ space. Thus, map construction is cubic in the stalk dimension, while memory grows quadratically. Each matrix-free diffusion step applies the node-mediated and face-induced components of the edge operator in

$$O\Big((\widetilde{M}+I_{\text{bnd}})d^2 + (N+\widetilde{M})d\Big)$$

time. Hence, when the MLP widths and d_r scale with d, the backbone costs

$$O\Big(N d_{\text{in}} d + (M+I_{\text{bnd}})d^3 + K\big[(\widetilde{M}+I_{\text{bnd}})d^2 + (N+\widetilde{M})d\big]\Big)$$

time and

$$O\Big((\widetilde{M}+I_{\text{bnd}})d^2 + (N+\widetilde{M}+I_{\text{bnd}})d\Big)$$

inference space. For fixed d and K, both costs are linear in the graph size and retained boundary incidences; no dense Laplacian is formed.

SheafCovPool. With three node/edge views, projected dimension d_p, pooling width d_{pool}, a hidden width shared by the projections ρ_q, the scoring networks g_q in Equation (29), and $\text{MLP}_{\text{graph}}$ in Equation (34), as well as N_{graph} graphs and C_{cls} classes, the readout costs

$$O\Big(\widetilde{M}d^2 + (N+\widetilde{M})(d + d_{\text{pool}}d_p + d_p^2) + N_{\text{graph}}(d_{\text{pool}}d_p^2 + d_{\text{pool}}C_{\text{cls}})\Big)$$

time and

$$O\Big((N+\widetilde{M})(d+d_p^2) + N_{\text{graph}}d_p^2\Big)$$

space. The $\widetilde{M}d^2$ term transports extended-edge states into their endpoint stalks. With fixed dimensions, SheafCovPool is linear in the extended graph size and constructs neither a dense node-to-node covariance matrix nor a coarsened graph.

Furthermore, we evaluated the computational efficiency of H2S-NN against several baselines on the ACM node-classification task using an NVIDIA A100 SXM4 Tensor Core GPU with 80 GB of VRAM. Table 11 reports the measured runtime, parameter count, and peak GPU-memory usage for the evaluated models.

Table 11: Efficiency comparison on ACM. Since HetSheaf's code is unavailable, we report the number of parameters from the original paper.

Model	Runtime (s/epoch)	Parameters	Memory (MB)
GCN	0.109	492K	140
GAT	0.321	928K	411
ChebNetII	0.400	633K	106
NSD	0.195	199K	989
CSNN	0.421	201K	760
HAN	0.448	618K	3,902
HINormer	0.674	3.17M	3,720
Hetero^{2}Net	0.662	555K	520
PSHGCN	1.242	505K	8,729
H2G-former	0.855	1.41M	2,337
HetSheaf	–	174K	–
H2S-NN	3.011	1.61M	9,260

Table 11 shows that H2S-NN required more parameters and memory, and had a higher runtime, than most models. This is because H2S-NN retains path-supported 2-cells and performs sheaf-Hodge diffusion on both node and edge cochains, which increases the computational cost. However, H2S-NN outperformed the compared baselines on most benchmarks, as shown in Table 1, Table 2, and Table 3. The trade-off between performance and efficiency motivates future work on reducing the computational and memory overhead of H2S-NN while preserving its predictive performance.